

Media Optimization: Apps Cheat Sheet

Baseline 2026-10-07. Image and video loading, BlurHash placeholders, caching, compression and uploads on Android, iOS, React Native, KMP and Flutter. Quick reference only: verify against the official docs and the full pages.

PICK THE VARIANT

- Choose the API variant whose width $\geq$ target pixels (dp x density); prefer AVIF/WebP, fall back to JPEG.
- WebP: Android lossy API 14, lossless/alpha API 18; iOS 14+. AVIF decode: Android 12 (API 31), iOS 16.
- HEIC: Apple platforms; convert on the server for everyone else.
- Always send width/height to reserve space; use the BlurHash as placeholder.

ANDROID IMAGES

- ```
coil-compose:3.6.3
coil-network-okhttp:3.6.3
// Application:
SingletonImageLoader.Factory
ImageLoader.Builder(ctx)
 .memoryCache {
 MemoryCache.Builder()
 .maxSizePercent(ctx, 0.25).build() }
 .diskCache {
 DiskCache.Builder()
 .directory(cacheDir.resolve("img"))
 .maxSizeBytes(256L shl 20).build() }
 .crossfade(true).build()
AsyncImage(model = url,
contentDescription = null,
placeholder = blurPainter,
contentScale = ContentScale.Crop)
```
- Coil 3 needs one coil-network-\* artifact. Glide 5.0.9: DiskCacheStrategy.ALL / AUTOMATIC; Compose via GlideImage (experimental). Fresco: legacy.
  - BlurHash: woltapp  
BlurHashDecoder.decode(hash, 32, 32) or vanniktech/blurhash 0.3.0 (KMP).

## ANDROID VIDEO

- ```
media3-exoplayer, -exoplayer-hls,
-exoplayer-dash, -ui, -datasource,
-transformer
ExoPlayer.Builder(ctx).build().apply {
    setMediaItem(MediaItem.fromUri(url))
    prepare() }
SimpleCache(File(cacheDir, "media"),
LeastRecentlyUsedCacheEvictor(200L shl 20),
StandaloneDatabaseProvider(ctx))
Transformer.Builder(ctx)
    .setVideoMimeType(VIDEO_H264).build()
retriever.getFrameAtTime(us,
OPTION_CLOSEST_SYNC)
```
- One SimpleCache per folder. The old ExoPlayer repo is archived: use Media3.
 - Release the player in onStop.

IOS IMAGES

Library	Key API
AsyncImage	no own cache, no prefetch, no downsampling: few small images only
Kingfisher	KFImage(url) with DownsamplingImageProcessor; ImagePrefetcher
Nuke	LazyImage(request:), ImagePipeline, ImagePrefetcher
SDWebImage	WebImage(url:), coder plugins

BlurHash (woltapp Swift):
UIImage(blurHash:size:punch:) at 32 x 32; cache the decoded image.

IOS VIDEO

- ```
VideoPlayer(player: AVPlayer(url: u))
let g = AVAssetImageGenerator(asset: a)
g.appliesPreferredTrackTransform = true
let (cg, _) = try await g.image(at: t)
AVAssetExportSession(asset: a,
presetName: AVAssetExportPreset1280x720)
URLSessionConfiguration.background(
withIdentifier: id)
session.uploadTask(with: req,
fromFile: url)
```
- HLS plays natively. Background uploads need a file, not Data.
  - shouldOptimizeForNetworkUse = true (index first).

## REACT NATIVE / EXPO

- ```
<Image source={{ uri: url }} placeholder=
{{ blurhash }}
    contentFit="cover" transition={250}
    cachePolicy="memory-disk" recyclingKey=
{url} />
await Image.prefetch(urls, { cachePolicy:
'disk' });
```
- expo-image 57: recommended; thumbhash placeholders too.
 - react-native-blurhash 2.1.3 for the bare Image.
 - react-native-fast-image 9.1.0: New Architecture through the interop layer; native component planned. Fork @d11/react-native-fast-image adds Fabric.
 - react-native-video 6.x: HLS native (API differs in v7).

KMP AND FLUTTER

- Coil 3 AsyncImage in commonMain; coil-network-ktor3 + a Ktor engine per platform.
- vanniktech/blurhash for shared decode.
- Video: expect/actual composable over Media3 and AVPlayer.
- Flutter:
CachedNetworkImage(memCacheWidth:), flutter_blurhash, video_player.

COMPRESS AND UPLOAD

- Server is the authority: re-validate and re-encode everything.
- Images: downsample on decode (ImageIO thumbnails, ImageDecoder target size), apply rotation, JPEG/WebP q 80.
- Video: Transformer / AVAssetExportSession to 720p H.264; HEVC stays HEVC unless asked.
- iOS compatible export:
preferredAssetRepresentationMode = .compatible.
- Upload to a pre-signed URL; background: URLSession background, WorkManager (long-running worker).
- Large files: S3 multipart, GCS resumable, Azure block, or tus. Keep the file until the API confirms; retry with backoff; refresh expired URLs.